

CARE to Compare, Wind Farm C — Plain-Language Companion

Experiments, dataset, the contamination question, and a route to federated learning

2026-09-21 · @Pranav

A companion to the technical journal. This one carries no derivations — it explains what was run, what the data is, whether the data-leak finding holds up under pressure, and what would have to change to train this federated.

The whole project in one page

The task. Wind turbines fail expensively. Their sensors record everything every 10 minutes. We want a system that notices a failure developing weeks before it stops the turbine.

The trick. Nobody labels sensor rows as "faulty" — that data does not exist. So instead we train a model to *copy* healthy data. Squeeze each reading through a narrow channel and force the model to rebuild it. On healthy data it rebuilds well, because it has seen that pattern thousands of times. On a degrading turbine it rebuilds badly, because it has never seen that pattern. How badly it rebuilds is the alarm signal.

The question we asked. Do you train one small model per turbine, or one big model for the whole farm? The big model gets 60 times more data, which normally wins. Utilities would much rather maintain one model than 58.

What we found. The small per-turbine models won. They found 16 of 27 faults; the fleet-wide model found 7. The fleet-wide model looked better on the false-alarm metric, but only because it barely fired at all.

Why. A model that has to describe 22 different turbines across all seasons has to call a very wide range of behaviour "normal". A fault on one turbine often looks like ordinary Tuesday operation on another, so it slips inside the boundary. A model that knows one turbine has a tight boundary and notices small deviations.

The thing we found by accident. While auditing the fleet-wide model, we discovered that a quarter of the rows it was being *tested* on had already been in its *training* data. Not because of a bug in our code — because the published benchmark ships the same

physical sensor readings inside several different event files. Anyone who pools this data across files is testing on their training set without knowing it.

What that is worth. Less than we expected. Stripping the affected rows moves the fleet-wide score by about 0.011, which is roughly the same size as the random variation between two identical runs. The duplication is definitely real; its effect on the score is small and not yet cleanly measured.

Where this goes next. Real wind operators will not pool raw SCADA data — it is commercially sensitive. Federated learning is the standard answer: train locally, share only model weights. Our result matters for that, because naive federated averaging produces one global model, which is structurally the fleet-wide model we just showed underperforms. The last two sections work through what to do about that.

The dataset

CARE to Compare is a public benchmark for early fault detection on wind turbine SCADA data. It covers three anonymised wind farms; we used Wind Farm C, because the operating-status labels are only usable for evaluation on Farms B and C.

Counts

Quantity	Value
Event datasets (CSV files)	58
Physical turbines behind them	22
Turbines contributing more than one event	19
Most events from a single turbine	6
Anomaly events / normal events	27 / 31
Rows per file	~55,000
Columns per file	~957
Sampling interval	10 minutes
Total healthy training rows across the farm	~2.56M
Total prediction (evaluation) rows	158,528

What one file looks like

Each file is one labelled window on one turbine, typically covering several months. Beyond the sensors it carries five bookkeeping columns.

Column	What it is
<code>time_stamp</code>	when the 10-minute average was taken
<code>asset_id</code>	which physical turbine
<code>train_test</code>	marks each row <code>train</code> or <code>prediction</code> — the file's own split
<code>status_type_id</code>	operating state; 0 and 2 mean normal operation
<code>id</code>	row index

The ~952 sensor columns come in quartets: every base sensor ships as `_avg`, `_min`, `_max` and `_std` over the 10-minute window. That is roughly 238 distinct physical sensors — power, wind speed, rotor and generator speed, a long list of bearing, gearbox, winding and nacelle temperatures, pitch and yaw angles, reactive power, and ambient conditions. We used only the `_avg` columns. The dataset FAQ warns that the min, max and std variants contain implausible values, and the averages are the native resolution of the signal anyway.

What the labels mean

A separate `event_info` table labels each of the 58 files as `anomaly` or `normal`, and for the anomalies gives an `event_start` and `event_end`. That window is the ground truth: flags inside it are hits, flags outside it on a normal file are false alarms. Important nuance — there are **no row-level fault labels**. The file tells you a fault happened somewhere in this window; it does not tell you which readings were abnormal. That is why the whole approach has to be unsupervised, and why the alarm threshold cannot be tuned against the answer.

The two properties that shaped everything

1. **Events are not independent.** 58 files, 22 turbines. Nineteen turbines appear more than once and one appears six times. The benchmark averages over events as if they were 58 independent trials; they are closer to 22.

2. Files overlap. The same physical readings appear in more than one event file, a documented side effect of the anonymisation procedure. This is the subject of the two sections below.

Every experiment we ran

Seven, counting the small ones. Only the first two are model training runs; the rest are audits and analyses, and most of the interesting output came from those.

#	Experiment	What it was for	What it returned
1	Arm A — 58 per-event models	The paper-faithful baseline	CARE 0.628, 16 of 27 faults found
2	Arm B — one fleet-wide model	Does more data win?	CARE 0.583, 7 of 27 faults found
3	Within-file fingerprint audit	Calibrate the leak detector	0.008% false-positive floor
4	Cross-file fingerprint audit	Measure real contamination	25.4% of evaluation rows contaminated
5	Leaked vs clean Accuracy split	What did contamination buy?	$\approx +0.011$ CARE
6	Leakage vs Coverage correlation	Did leakage hide faults?	No effect found; test underpowered
7	Repeat run of Arm B	How noisy is one run?	± 0.013 CARE

1. Arm A — one model per event

For each of the 58 files in turn: read it, keep only the healthy training rows, fit a scaler on those rows, train a fresh autoencoder, set an alarm threshold from held-out healthy rows of that same file, score that file's own evaluation window, throw everything away, move to the next file.

Nothing crosses a file boundary. About 38,000 training rows per model, 113 minutes for all 58 on one GPU.

Result: CARE 0.628. Found 16 of 27 faults, with 10 false alarms across the 31 normal files.

2. Arm B — one model for the farm

Two passes. First, read all 58 files and pool every healthy training row into one pile — 2.56M rows. Hold out three whole turbines for validation, fit one scaler, train one model, set one threshold. Second, re-read all 58 files and score every evaluation window with that single model.

Everything else is the identical code. Only the training strategy differs, which is what makes the comparison fair.

Result: CARE 0.583. Found 7 of 27 faults, with 1 false alarm. Faster to train than Arm A (25 minutes) despite 60x the data, because it is one model instead of 58.

3. The control audit — calibrating the leak detector

Before trusting any leak measurement you need to know how often the detector cries wolf. So we ran it on Arm A, where leakage is *impossible* by construction — training and evaluation rows come from disjoint halves of the same file.

It still reported 13 matches out of 158,528 rows. Those cannot be leaks, so they are the detector's own error rate: 0.008%. Two genuinely different timestamps can have identical readings when the turbine is parked and every channel is flatlined.

This experiment produced no headline number, and it is the most important one in the list. Everything in experiment 4 is only interpretable against it.

4. The real audit — measuring contamination in Arm B

Same detector, run across files: is any evaluation row in Arm B identical to a row somewhere in its pooled training pile?

Result: 40,216 of 158,528 evaluation rows — 25.4% — across 27 of the 58 events. Against a 0.008% error floor, that is about 3,000 times the chance rate.

5. What the contamination was worth

Split the 31 normal events by whether they were contaminated, then compare.

Contaminated normal events scored 0.9966 on the false-alarm metric; clean ones scored 0.9398. Working backwards from the overall 0.967 implies roughly 15 of 31 normal events were substantially contaminated. Rescoring on clean events only drops the fleet-wide CARE from 0.583 to about 0.572.

Result: contamination is worth roughly +0.011 CARE. Real, but small — and see the validity section, because this number has a problem.

6. Did contamination hide faults?

Our pre-registered expectation: if the model trains on anomalous rows, it learns to rebuild them, so it should *fail* to flag them. More contamination on a fault event should mean worse detection.

Result: no. The correlation came out slightly positive (+0.349) and not significant ($p = 0.075$).

The reason is visible once you look at the distribution rather than the statistic: contamination sits almost entirely in the *normal* events. Across the 27 fault events the median contamination is 0.00% and only one exceeds 50%. There was nothing to correlate against. The hypothesis was not refuted — it was untestable on this data, which is a different and more honest conclusion.

7. How much does a single run mean?

We ran Arm B twice with identical code and the identical seed. GPU reductions are not bit-reproducible, so the runs diverge.

Result: CARE 0.570 and 0.583, finding 6 then 7 faults. So ± 0.013 is the noise floor on any single number in this study. Any gap smaller than that is not a finding.

How we tested for contamination

The idea

Give every row a fingerprint, then ask whether any evaluation row's fingerprint already appears in the training pile.

A row's fingerprint is a hash of its 238 sensor readings, rounded to six decimals. Three details make it work:

- **Only sensor values go into the hash.** Not the timestamp, not the turbine id, not the train/test marker. Those are exactly the fields the anonymisation procedure alters, so including them would hide the duplicates. Hashing the physics rather than the labels is what lets the audit see through the anonymisation.

- **Rounding to six decimals** absorbs float storage noise without merging genuinely different readings.
- **It is cheap.** Set membership is instant, so auditing 2.5M training rows against 158k evaluation rows takes seconds. There is no cost reason for anyone to skip this.

```
flowchart LR
    A[every training row] --> B[hash the 238<br/>sensor values]
    B --> C[(training<br/>fingerprint set)]
    D[every evaluation row] --> E[hash the same way]
    E --> F{already in<br/>the set?}
    C --> F
    F -->|yes| G[contaminated]
    F -->|no| H[clean]
```

Why we ran a control first

A match does not necessarily mean copying. Two different moments can produce identical readings if nothing is moving — a parked turbine in a shutdown reports the same flatlined values every ten minutes. Those are *semantic* false positives, and no amount of theory tells you how often they happen. It depends on how much of the record is downtime.

So we measured it. Arm A trains and evaluates on disjoint halves of the same file, so contamination there is impossible by construction, not by inspection. Every match the detector reports in Arm A is therefore a false positive, by definition.

It reported 13 in 158,528 rows: **0.008%**. That is the detector’s noise level, measured rather than assumed.

	Arm A (control)	Arm B (measured)
Evaluation rows tested	158,528	158,528
Matches found	13	40,216
Rate	0.008%	25.4%
Events affected	3	27 of 58

The structural checks

A 3,000x excess is suggestive but not conclusive on its own. Four further checks distinguish copying from coincidence:

- 1. **Byte-identical across 226 columns.** Not similar — identical, in every channel at once. Coincidence does not align 226 sensors.
- 2. **A single rigid offset per file pair.** If file A's row 4,000 matches file B's row 12,000, then A's 4,001 matches B's 12,001, and so on. The matches are a block copy at a fixed displacement, not scattered hits.
- 3. **Long contiguous runs.** Median longest unbroken run is 720 records — five continuous days of identical readings.
- 4. **Thirty control pairs from different turbines returned nothing.** The detector does not fire on two turbines merely operating similarly.

What the benchmark itself says

The dataset FAQ independently states that overlaps across different event CSV files are an expected side effect of the anonymisation procedure, and that event files may share timestamp ranges even when they represent different events.

So this is not a discovery in the sense of finding something the authors hid. It is documented. What appears to be missing from the field is anyone *measuring* it and reporting what it does to their score.

Is the data-leak story valid?

Partly. The duplication is solidly established. The claim about what it *does to the score* is not, and we should stop repeating the +0.011 figure as though it were measured.

Splitting the story into its two halves makes this clear.

Claim	Verdict
The published files contain duplicated rows	Holds. Multiple independent lines of evidence, plus the FAQ.
Pooling training data therefore trains on evaluation rows	Holds. Follows mechanically from the above.

Claim	Verdict
25.4% of Arm B's evaluation rows were contaminated	Holds , as a lower bound.
Contamination was worth +0.011 CARE to Arm B	Does not hold yet. See below.
Contamination suppressed fault detection	Not supported. We correctly did not claim it.

What is solid

The existence claim is about as well-supported as an observational finding gets. Byte-identical rows across 226 columns, aligned at a rigid offset, in five-day contiguous runs, with thirty cross-turbine controls returning nothing, against a measured 0.008% false-positive floor — and the dataset’s own documentation saying the overlap is expected. None of those four evidence lines depends on the others. That is what makes it robust.

The mechanical consequence is not really an empirical claim at all. If the same row is marked `train` in one file and `prediction` in another, then pooling by definition puts evaluation rows in the training set. There is nothing to dispute.

The problem with the +0.011

It is the same size as our noise floor. Two identical runs of Arm B differed by 0.013 CARE. We are reporting a contamination effect of 0.011 from a single run of each condition. That effect is *inside* the measurement error, and we should say so plainly rather than quoting it as a result.

It is an inference, not a count. "Roughly 15 of 31 normal events are substantially leaked" was back-calculated from the Accuracy arithmetic, not read off the leakage table. The per-event contamination percentages are already saved in `leakage_pooled.csv` — the events should simply be counted. This is an afternoon of work and it would replace an estimate with a fact.

It only prices one of two channels. We measured what contamination did to the false-alarm metric on normal events. We could not measure what it did to fault detection, because there was almost no contamination on fault events to vary. So +0.011 is the effect on one of four CARE components, presented as the total.

Two further caveats

25.4% is a floor, not the true rate. The fingerprint only catches exact duplicates. If the anonymisation resampled, jittered or rescaled some copies, those rows are contaminated but invisible to this method. The real contamination is at least 25.4% and could be higher.

The control is a good null, not a perfect one. The 0.008% floor came from within-file comparison. Cross-file comparison draws from a larger and differently composed pool of rows, so the true semantic-collision rate there is not exactly 0.008%. Given the effect is 3,000x, this does not change the conclusion — but it means the floor is an estimate too.

What would settle it

One run. Train a third arm: pooled, exactly like Arm B, with every contaminated row removed from the training pile before fitting.

That gives three points instead of two, and each comparison isolates one variable:

Comparison	Isolates
Arm B vs Arm C	the effect of contamination, directly measured
Arm A vs Arm C	the effect of per-turbine specialisation, cleanly

Run it at three seeds per arm and both effects come with error bars. Until then, the honest framing for any talk or paper is:

The benchmark contains substantial cross-file duplication, which we measured at 25.4% of evaluation rows against a 0.008% false-positive floor. Its effect on the composite score appears small, but our current estimate sits within single-run variance and should not be treated as measured.

Preparing this dataset for federated learning

Why federated is the right next step

The premise of Arm B — pool everyone's raw SCADA into one pile — is not available in the real world. Turbine sensor data is commercially sensitive: it reveals availability, output and maintenance history, and it is usually contractually locked between operator, OEM and

asset owner. That is the classic motivation for federated learning: each party trains locally and shares only model weights, never rows.

What makes this project unusually well-placed to study that is that **the two arms already bracket the federated case**.

Setting	What it means	Our result
Fully local	each turbine trains alone, no collaboration	Arm A, CARE 0.628
Federated	train locally, share weights	not yet run
Fully centralised	pool all raw data	Arm B, CARE 0.583

Federated learning is normally sold as "nearly as good as centralised, without sharing data". Here, centralised is the *worse* endpoint. So the interesting question is not whether federated approaches Arm B — it is whether federated can beat both, by sharing statistical strength without collapsing into one over-general model.

Step 1 — define the client

`asset_id` is the natural client boundary: 22 turbines, 22 clients. That makes this cross-silo federated learning (few clients, lots of data each, all reliably online), not cross-device.

A realistic variant is to group turbines into 3–5 synthetic "operators" and federate across those. That better matches the commercial reality, where the privacy boundary is the company, not the machine. Both are worth running; the turbine version is the cleaner experiment.

What you must **not** do is make each event file a client. Those overlap.

Step 2 — deduplicate, and note the convenient accident

The duplication is *within* turbines, across their event files — the 30 cross-turbine control pairs found nothing. That is a lucky property: partitioning by `asset_id` puts every copy of a row on the same client.

So turbine-level partitioning removes cross-client leakage automatically. Arm B's validation split already did this, for the same reason.

Within a client you still want to drop the duplicates, or that turbine's data is silently weighted by how many event files it happens to appear in. Deduplicating on the row fingerprint before training fixes it:

```
for each client k:
    rows_k = concat(all event files with asset_id == k)
    rows_k = drop_duplicates(rows_k, on = fingerprint)
```

This also changes the client weights in the aggregation, which matters for FedAvg.

Step 3 — agree a feature schema without sharing data

Every client's model must have identical input dimensions or the weights cannot be averaged. Two things currently break that:

- **The NaN-drop rule.** We drop any feature more than 30% missing *in training*. Run per client, different clients drop different columns and the models become unaveragable. Fix: one federated round where each client reports per-column missing counts, the server unions the drop list, and everyone uses it.
- **Angle detection.** The sin/cos treatment and the range check must be decided once, globally, from metadata — never per client from local ranges.

Neither leaks anything meaningful. Column missing-counts are schema facts, not observations.

Step 4 — decide where standardisation happens

Our pipeline fits mean and standard deviation on training rows. Federated gives two options:

Global scaler via secure aggregation. Each client sends count, sum and sum-of-squares per column; the server combines them:

$$\mu_j = \frac{\sum_k n_k \mu_{k,j}}{\sum_k n_k}, \quad \sigma_j^2 = \frac{\sum_k n_k (\sigma_{k,j}^2 + \mu_{k,j}^2)}{\sum_k n_k} - \mu_j^2$$

Three scalars per column per client, aggregated under secure sum so the server never sees an individual client's statistics.

Local scalers. Each client standardises against its own distribution. This is arguably *better* here, and it is the more interesting option: it removes inter-turbine offsets (calibration drift, siting differences) before the shared model ever sees them, so the global weights learn the common physics rather than 22 different sensor biases.

Recommendation: run local scalers as the default and global as an ablation. Local is both more private and more likely to work.

Step 5 — keep the threshold local, always

This one is not a design choice, it is a requirement. The alarm threshold is the 99th percentile of reconstruction error on that client's own held-out healthy rows. It depends on local noise, local sensor calibration and local climate.

A single global threshold is exactly what Arm B used, and part of why it flagged so little. Federated learning gives you the chance to share the model and keep the threshold local — which is Arm A's best property combined with Arm B's data advantage. That combination is the core of the proposal.

Step 6 — make evaluation federated too

CARE needs event labels and per-event confusion counts. No raw rows need to move: each client computes its own per-event true positives, false positives, false negatives and true negatives, and reports those integers. The server assembles Coverage, Earliness, Reliability and Accuracy from the totals.

The criticality counter is per-event and sequential, so it runs entirely client-side. Nothing about the scoring forces data to centralise.

The federated roadmap

The prediction our results already make

Standard federated averaging trains local copies, then averages the weights:

$$\theta^{t+1} = \sum_{k=1}^K \frac{n_k}{n} \theta_k^{t+E}$$

The fixed point of that procedure approximately minimises the weighted average of every client's loss — which is, near enough, the objective Arm B optimised directly. **So naive FedAvg should reproduce Arm B's failure mode:** one global model, one wide manifold, faults that look normal somewhere in the fleet and slip through.

That is a testable prediction and it is worth stating before running the experiment. If federated CARE lands near 0.58 rather than 0.63, the result is not a disappointment — it is confirmation that the problem is the shared-model objective, not the data-sharing mechanism.

What should work instead

Personalised federated learning: share what generalises, keep local what does not.

Component	Shared or local	Reasoning
Encoder weights	shared	The physics of how SCADA channels relate is common to all turbines.
Decoder weights	local fine-tune	Reconstruction has to hit this turbine’s calibration and siting.
Scaler	local	Removes per-turbine sensor bias before the shared model sees it.
Threshold	local, always	A quantile of this turbine’s own error distribution.

The mechanism is FedAvg for the shared layers plus a few epochs of local fine-tuning each round. Each client ends with its own model, initialised from fleet-wide knowledge — Arm A’s specificity with Arm B’s sample size. That is the combination the two arms suggest should win, and nobody has tested it on this benchmark.

Handling non-IID clients

Turbines differ in siting, wake exposure, control configuration and age, so client distributions are not identical. That causes client drift: local models wander toward local optima and averaging them lands somewhere useful to nobody.

FedProx is the standard mitigation — add a penalty keeping local weights near the current global ones:

$$F_k^{\text{prox}}(\theta) = F_k(\theta) + \frac{\mu}{2} \|\theta - \theta^t\|^2$$

μ then becomes an explicit dial between the two arms. At $\mu \rightarrow \infty$ every client is pinned to the global model, which is Arm B. At $\mu = 0$ with enough local epochs, clients barely interact, which approaches Arm A. Sweeping μ traces the whole curve between our two data points, and the shape of that curve is the paper.

Communication cost is a non-issue

The model is 90,731 parameters — about 363 KB in float32. With 22 clients and 100 rounds, total traffic is under 1.6 GB, and rounds are minutes apart rather than milliseconds. This is one of the rare federated settings where you can ignore bandwidth entirely and focus on the statistics.

Privacy, honestly stated

Weight updates are not automatically private — gradients can leak training data. Two additions, in order of cost:

- **Secure aggregation** for both the scaler statistics and the weight updates, so the server sees only sums. Cheap and standard.
- **Differential privacy** via clipped, noised updates. This costs accuracy, and on an anomaly detector it may cost detection specifically, since the tail behaviour that matters is exactly what noise blurs. Measure the CARE cost per privacy budget rather than assuming it is affordable.

Worth being straight about this in any write-up: federated learning removes raw-data sharing, which is the commercial blocker. It does not by itself provide a formal privacy guarantee.

Phased plan

Phase	Work	Output
0	Arm C (pooled, deduplicated) at 3 seeds	Clean baseline; settles the contamination question
1	Repartition by <code>asset_id</code> , dedupe within client, federated schema round	22-client dataset ready for FL
2	Naive FedAvg, local thresholds	Tests the prediction above
3	Personalised FL: shared encoder + local fine-tune	The candidate winner
4	Sweep FedProx μ from 0 to large	The curve between Arm A and Arm B
5	Add secure aggregation, then DP, measuring CARE cost at each step	Deployment-realistic numbers

Phase 0 is a prerequisite, not an option — federating contaminated data reproduces the contamination inside the clients.

The paper this becomes

The framing writes itself once Phase 4 is done: *a single benchmark where centralised training is worse than isolated training, and federated personalisation recovers the gap.* That inverts the usual federated-learning narrative, it comes with a mechanism (manifold width) rather than just a number, and it ships a reusable contamination audit as a secondary contribution.

Verification results from the dedicated overlap notebook

A separate notebook, `dataleaktest.ipynb`, exists purely to attack the duplication finding. It runs eight tests plus a trivial-match audit, and it comes with stored outputs — unlike the two arm notebooks, whose cells were cleared.

It substantially upgrades the evidence. Everything in the earlier sections was established with a single fingerprint hash; this notebook confirms the same conclusion through independent routes, several of which were designed to fail if the finding were spurious.

Setup

Files	58, 238 avg columns
Turbines	22, of which 19 hold more than one sub-dataset
Within-turbine pairs tested	64
Fingerprint columns (used to <i>find</i> alignment)	12
Holdout columns (used to <i>verify</i> , disjoint)	20
Full-width test columns	226

The split between fingerprint and holdout columns is the important design choice. Alignment is found using 12 columns, then confirmed on 20 columns that played no part in finding it. A method that manufactured its own matches would fail on the holdout set.

The eight tests

Test	What it checks	Result
1 — Offset structure	Are matches at one rigid displacement?	<code>off_std = 0.00</code> on every duplicate pair
2 — Holdout columns	Do disjoint columns agree?	Worst holdout match 1.000000
3 — Full width	Do all remaining columns agree?	226 columns, worst match 1.000000
4 — Contiguity	Are matches blocks or scatter?	Up to 603 separate runs per pair
5 — Cross-turbine control	Does it fire on different turbines?	30 pairs, 0 false positives
6 — Wrong-delta control	Does it fire at the wrong alignment?	1.000 at offset 0, 0.0078 at +10 min
7 — H1 vs H2	Shared date ranges, or shared data?	49 pairs require shared data
8 — Precision sweep	Are matches precision-dependent?	Count flat from 1 to 6 decimals

Verdict: 49 of 64 within-turbine pairs verified as duplicates, 0 pairs failed verification.

Tests 5 and 6 are the ones that could have sunk the finding. Test 6 in particular is the cleanest: shift a confirmed duplicate pair by a single 10-minute step and agreement collapses from 1.000 to 0.008. A flat profile there would have meant the columns were constant and the whole result meaningless.

Test 7 also settles a question the FAQ leaves open. The FAQ says event files may share timestamp *ranges*, which would be innocuous. Twenty-four pairs are explicable that way. Forty-nine are not — they require shared measurements, not merely shared dates. The observed time shifts are exactly -365 days, 0 and $+365$ days, which is a per-file anonymisation offset rather than anything physical.

The trivial-match audit

The obvious objection to "the rows are identical" is that you might be matching zero-padding and missing values. The notebook tests this directly by restricting to live cells — non-zero, non-NaN — and re-checking.

Quantity	Value
Live cells compared	79,865,570
Exact matches among them	79,865,570
Worst pair match rate	1.000000
Distinct values involved (largest pair)	262,070
Share of cells that are trivial	~10%

Every one of 79.8 million live cells matched, across a quarter of a million distinct values. The agreement cannot be an artefact of padding.

The zero patterns then agree too, at 1.000000, with zero cells blanked in one file but not the other. If two files had been exported independently you would expect their missing-value substitutions to disagree occasionally. Identical blanking means the zeros were already in a shared source record — which is separate evidence that the files were cut from one continuous series.

Turbine-level redundancy, and what it corrects

The measurement

The visualisation cell computes a time-free fingerprint universe for one turbine. It is hardcoded to `TURBINE = 53`, which holds 6 sub-datasets — the most of any turbine on the farm.

File	Rows	Unique fingerprints	Raw span
1.csv	53,569	47,259	2022-10-01 → 2023-10-16
16.csv	53,568	45,215	2022-12-23 → 2024-01-07
20.csv	54,001	49,756	2022-07-12 → 2023-07-30

File	Rows	Unique fingerprints	Raw span
35.csv	52,615	47,171	2022-09-22 → 2023-09-30
60.csv	54,433	43,645	2022-02-14 → 2023-03-03
76.csv	52,086	43,469	2022-01-06 → 2023-01-11
total	320,272		
union across all six		80,256	

So for turbine 53:

- **74.9% redundancy** against all rows, a **4.0x** inflation factor
- 71.0% against per-file-unique rows
- 13.7% of rows are duplicated *within* individual files, before any cross-file overlap enters

Turbine 53 is the worst case, not the typical one — three turbines hold a single file and have 0% redundancy by construction. A fleet-wide average well below 75% is expected. **The fleet figure is not yet measured**, because the cell runs for one turbine and the fleet-view cell aggregates pair counts rather than redundancy fractions. Looping `TURBINE` over `sorted(turbines)` and collecting `len(union) / sum(rows)` per turbine would close this in one run.

Contamination is not evenly spread

The directed table for turbine 53 shows how much of each file's evaluation data sits in another file's training data:

File	Prediction rows	Contaminated	%	Contaminating files
1.csv	2,121	2,121	100.0	16, 76
20.csv	2,492	2,492	100.0	1, 16, 35, 76
60.csv	2,376	2,376	100.0	1, 16, 20, 35, 76
35.csv	944	58	6.1	1, 16, 76
16.csv	445	0	0.0	—
76.csv	668	0	0.0	—

This is the most important correction in this section. Our 25.4% farm-wide figure reads like a uniform 25% haze over everything. The real shape is bimodal: some events are **totally** contaminated and others are perfectly clean. A mean is a poor summary of that distribution, and any per-event analysis should treat contamination as a near-binary flag rather than a continuous percentage.

Corrections to earlier sections

These supersede what is written above; the earlier text is left in place so the revision is visible.

Federated prep, within-client deduplication. That section estimates a 1.7x inflation in client sample counts, derived from an assumed ~40% redundancy. On the measured turbine-53 figure it is **4.0x**. This makes the FedAvg weighting problem considerably worse than described: client weights n_k / n would over-count turbines in proportion to how many event files they happen to appear in, which is an artefact of dataset packaging with no physical meaning. Deduplicating within client is not a tidiness step — without it the aggregation is weighted by a packaging accident.

Validity section, the lower-bound caveat. The caveat stands and gets stronger. 25.4% counts pooled prediction rows found in the pooled training corpus; the turbine-level union redundancy is a different and much larger quantity, and the fingerprint still only catches exact duplicates.

Validity section, the existence claim. It was already the solid half of the story. It is now established through eight independent tests with two negative controls, and the trivial-zeros objection is closed. This is no longer a caveated finding.

What still is not settled

None of this touches the weak half. The effect of contamination on the CARE score is still an inference of +0.011 from single runs, still inside the ± 0.013 noise floor, and still unmeasured on the detection side. Stronger evidence that duplication *exists* does not make our estimate of what it *cost* any more precise. Arm C at three seeds remains the thing that settles it.